

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

FUNDAMENTAL PROGRAMMING NOTES: C++

Composed by: HAFIZ MUHAMMAD ASAD MUSTAFA | LMS ID: S23NDOCS1M01042 | Affiliation: President of IUB CSSC

Table of Contents

1. INTRODUCTION TO C++	2
2. DATA TYPES AND VARIABLES	2
3. INPUT/OUTPUT OPERATIONS	3
4. OPERATORS	5
5. CONTROL STRUCTURES	5
6. FUNCTIONS	7
7. ARRAYS	9
8. POINTERS	10
9. STRINGS	11
10. STRUCTURES (struct)	12
11. OBJECT-ORIENTED PROGRAMMING (OOP) FUNDAMENTALS	12
Exercise:	13
Short Questions:	13
Long Questions:	30

Thank you for reading these notes. If you find them helpful, **share them with others**, because *sharing is caring* and **knowledge grows when it is shared**.

If you have any questions, suggestions, or need further assistance, feel free to contact me.

Contact Information:

Email: hafizasad2554@gmail.com

Phone: 0303-2554632

1. INTRODUCTION TO C++

What is C++?

C++ is a general-purpose, compiled, object-oriented programming language developed as an enhancement of C language. It combines high-level and low-level programming features.

History and Features

- Developed by Bjarne Stroustrup at Bell Labs in 1979
- Originally called "C with Classes"
- Features: **Object-oriented, platform dependent, rich library support, speed and efficiency**

Structure of a Basic C++ Program

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!";
    return 0;
}
```

2. DATA TYPES AND VARIABLES

Definition & Importance

A **data type** specifies the kind of data a variable hold, and a **variable** is a named memory location that stores that value. Types are critical because they determine the memory size, internal representation, and the validity of operations (**Type-Checking**).

Primitive Data Types

Category	Types	Description
Integers	int, short, long, long long	Whole numbers (e.g., 4 bytes for int).
Floating Point	float, double, long double	Real numbers (with decimal points).

Character/Boolean	char (1 byte), bool (true/false).	
Absence	void	Used mainly for functions that return nothing.

Primitive Data Types

- **int**: Integer numbers (4 bytes)
- **float**: Single-precision floating point (4 bytes)
- **double**: Double-precision floating point (8 bytes)
- **char**: Single character (1 byte)
- **bool**: Boolean values (true/false)

Variable Declaration and Initialization

```
int age = 25;  
float height = 5.9f;  
char grade = 'A';  
bool isEmployed = true;
```

Constants and Casting

- **Constants**: Use **const** or **constexpr** (preferred for compile-time constants).
- **Implicit Conversion**: Automatic type change by the compiler (e.g., int to double).
- **Explicit Casting**: Manual conversion using syntax like `static cast<T>(expr)` or `(T)expr`.

Example Casting:

```
// Implicit casting  
int intVar = 10;  
double doubleVar = intVar;  
  
// Explicit casting  
double price = 19.99;  
int intPrice = (int)price;
```

3. INPUT/OUTPUT OPERATIONS

Standard Streams

I/O is managed via the `<iostream>` library.

C++ By HAFIZ MUHAMMAD ASAD MUSTAFA

- **std::cin**: Standard input (keyboard) using the **extraction operator (>>)**.
- **std::cout**: Standard output (console) using the **insertion operator (<<)**.
- **std::cerr**: Unbuffered standard error stream.

Input Handling

- **Basic Input**: `cin >> var` automatically skips whitespace.
- **Input with Spaces**: Use **std::getline(cin, str)** (requires `<string>`) to read an entire line, including spaces.

Formatting Output

Requires the `<iomanip>` header. Manipulators control output appearance:

- **fixed / setprecision(N)**: Forces decimal notation and sets the number of digits after the decimal.
- **setw(N)**: Sets the minimum field width for the next output item.

Using cout and cin

```
#include <iostream>
using namespace std;

int main() {
    string name;
    int age;

    cout << "Enter your name: ";
    cin >> name;
    cout << "Enter your age: ";
    cin >> age;

    cout << "Hello " << name << ", you are " << age << " years old.";
    return 0;
}
```

Manipulators

```
#include <iomanip>
cout << fixed << setprecision(2) << 3.14159;
cout << setw(10) << "Hello";
```

4. OPERATORS

Categories and Function

Category	Operators	Purpose
Arithmetic	+, -, *, /, %	Basic math, including modulo (%) for integer remainder.
Relational	==, !=, <, >, <=, >=	Produce a bool result (comparison).
Logical	&& (AND), (OR), ! (NOT)	To Combine more than one Boolean Type results and give only one answer,
Assignment	=, +=, -=, *=, etc.	Compute and assign in one step (e.g., x += 3; is x = x + 3;).
Increment/Decrement	{++i, --i (Pre)} {i++, i-- (Post)}	Add or subtract 1 from the variable.

Precedence and Ternary

- **Precedence:** Determines for which operation is evaluated first (e.g., multiplication before addition).
- **Ternary Operator:** condition ? expr1 : expr2 provides a concise if-else expression.
- **Operator Precedence**

1. Parentheses
2. Multiplication/Division
3. Addition/Subtraction

5. CONTROL STRUCTURES

Selection (Decision Making)

- **if / if-else / if-else if-else:** Executes code based on boolean conditions.
- **switch:** Executes one of many code blocks based on the value of an integral expression. Requires **break** to prevent **fall-through**.

Iteration (Loops)

Loop Type	Usage	Execution Guarantee
for	When the number of iterations is known or easily calculated.	Condition must be true to run.
while	Condition-first loop; iterates as long as the condition is true.	Condition must be true to run.
do-while	Condition-last loop.	Runs at least once .

Jump Statements

- **break:** Immediately exits the nearest enclosing loop or switch block.
- **continue:** Skips the remaining code in the current loop iteration and proceeds to the next iteration.
- **return:** Exits the function.

Conditional Statements

```
// if-else
if (marks >= 50) {
    cout << "Pass";
} else {
    cout << "Fail";
}

// switch
switch(day) {
case 1: cout << "Monday"; break;
default: cout << "Invalid";
}
```

Looping Constructs

```
// for loop
for (int i = 1; i <= 5; i++) {
    cout << i << " ";
}

// while loop
while (i <= 5) {
    cout << i << " ";
    i++;
}

// do-while loop
do {
```

```
cout << i << " ";  
i++;  
} while (i <= 5);
```

break and continue.

```
for (int i = 1; i <= 10; i++) {  
    if (i == 6) break; // exit loop  
    if (i % 2 == 0) continue; // skip even numbers  
    cout << i << " ";  
}
```

6. FUNCTIONS

Definition and Structure

A function consists of a **Prototype** (declaration—signature to the compiler) and a **Definition** (the implementation).

Parameter Passing

- **Pass-by-Value:** The function receives a **copy** of the argument; the original variable remains unchanged.
- **Pass-by-Reference:** Using a reference (&) in the parameter list allows the function to directly access and **modify the original variable**.

Advanced Function Concepts

- **Function Overloading:** Defining multiple functions with the same name but different **parameter lists** (different signatures).
- **Recursion:** A function calling itself. Requires a **base case** to prevent infinite execution.
- **Function Pointers:** Variables that store the memory address of a function.

Defining and Calling Functions

```
#include <iostream>
```

```
using namespace std;

// Function definition
int add(int a, int b) {
    return a + b;
}

int main() {
    // Function call
    int result = add(5, 3);
    cout << "5 + 3 = " << result;
    return 0;
}
```

Function Prototypes

```
int multiply(int a, int b); // Prototype

int main() {
    int result = multiply(4, 5);
    return 0;
}

int multiply(int a, int b) { // Definition
    return a * b;
}
```

Pass by Value vs. Pass by Reference

```
// Pass by value (copies argument)
void swapByValue(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

// Pass by reference (works with original)
void swapByReference(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}
```

Function Overloading

```
int add(int a, int b) {
    return a + b;
}

double add(double a, double b) {
    return a + b;
}

string add(string a, string b) {
    return a + b;
}
```

Recursion

```
int factorial(int n) {  
    if (n == 0 || n == 1) return 1;  
    else return n * factorial(n - 1);  
}
```

7. ARRAYS

Definition and Indexing

An array is a fixed-size collection of items of the **same type** stored contiguously. Access is via a **0-based index**.

- **Multi-Dimensional Arrays:** Used to model matrices. E.g., `mat[1][2]` accesses the element at row 1, column 2.

Critical Exam Point: Array Bounds

Accessing indices outside the array's declared size results in **undefined behavior**; C++ does not prevent this automatically.

Arrays and Functions

When passed to a function, a raw array **decays to a pointer** to its first element. (Modern C++ uses `std::vector` for dynamic size).

One-Dimensional Arrays

```
int numbers[5] = {10, 20, 30, 40, 50};  
  
// Accessing elements  
for (int i = 0; i < 5; i++) {  
    cout << numbers[i] << " ";  
}
```

Two-Dimensional Arrays

```
int matrix[2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}  
};  
  
// Accessing elements  
cout << matrix[1][2]; // Output: 6
```

8. POINTERS

Definition and Syntax

A pointer is a variable that stores the **memory address** of another variable.

Operator	Symbol	Purpose
Address-of	&	Retrieves the memory address of a variable (int *p = &x;).
Dereference	*	Accesses the value stored at the address pointed to (cout << *p;).

Pointers and Arrays

The array name is interchangeable with a pointer to the first element. Array indexing arr[i] is equivalent to **Pointer Arithmetic**: *(arr + i).

- **Null Pointer**: `nullptr` signifies a pointer that doesn't point to a valid memory location.
- **Dynamic Memory**: Managed manually using **new** (allocation on heap) and **delete** (deallocation).

Pointer Declaration and Initialization

```
int number = 42;
int *ptr = &number;

cout << "Value: " << *ptr; // 42
cout << "Address: " << ptr; // memory address
```

Address-of (&) and Dereference (*at) Operators.

```
int x = 10;
int *p = &x;

cout << *p; // 10
*p = 20; // modifies x to 20
```

Pointers and Arrays

```
int arr[3] = {10, 20, 30};
```

```
int *ptr = arr;  
  
cout << *(ptr + 1); // 20
```

Null Pointers

```
int *ptr = nullptr;  
if (ptr == nullptr) {  
    cout << "Pointer is null";  
}
```

9. STRINGS

Types and Preference

- **C-Style Strings:** Null-terminated (`\0`) arrays of char. Prone to errors.
- **`std::string`:** Dynamic, safe, and feature-rich container (`<string>`). **Always prefer `std::string`.**

`std::string` Operations

Supports operations like concatenation (+), getting the length (`.size()`), and extracting parts (`.substr()`). Use **`getline(cin, s)`** for input containing spaces.

C-style Strings

```
char str1[] = "Hello";  
char str2[10] = "World";  
  
cout << strlen(str1); // 5  
strcat(str1, str2); // Hello World
```

`std::string` Class

```
#include <string>  
string s1 = "Hello";  
string s2 = "World";  
string s3 = s1 + " " + s2;  
  
cout << s3.length(); // 11  
cout << s3.substr(0, 5); // Hello
```

10. STRUCTURES (struct)

Definition and Usage

A struct groups related data members of distinct types under a single name (a record). Default member access is public.

Defining and Using Structures

```
struct Student {  
    string name;  
    int rollNumber;  
    float marks;  
};  
  
int main() {  
    Student s1;  
    s1.name = "Mr. Asad";  
    s1.rollNumber = 101;  
    s1.marks = 85.5;  
  
    Student s2 = {"Mr. Ahmad", 102, 78.0};  
    return 0;  
}
```

Passing Struct to Functions

For efficiency, structs are generally passed by **reference** (const Person& p) to avoid copying potentially large data sets.

11. OBJECT-ORIENTED PROGRAMMING (OOP) FUNDAMENTALS

Core OOP Concepts

- **Class:** Blueprint for objects
- **Object:** Instance of a class
- **Encapsulation:** Bundling data and methods
- **Inheritance:** Creating new classes from existing
- **Polymorphism:** Same interface, different implementations

Basic Class Definition

```
class Car {  
private:  
    string brand;
```

```
    string model;
    int year;

public:
    void setDetails(string b, string m, int y) {
        brand = b;
        model = m;
        year = y;
    }

    void displayDetails() {
        cout << "Brand: " << brand << endl;
        cout << "Model: " << model << endl;
        cout << "Year: " << year << endl;
    }

};

int main() {
    Car car1;
    car1.setDetails("Toyota", "Camry", 2022);
    car1.displayDetails();
    return 0;
}
```

Constructor and Destructor

```
class Book {
public:
    // Constructor
    Book(string t, string a) {
        title = t;
        author = a;
    }

    // Destructor
    ~Book() {
        cout << "Object destroyed";
    }

private:
    string title;
    string author;
};
```

Exercise:

Short Questions:

Q.1 : What is a general-purpose, compiled language?

Ans: A general-purpose language is designed for a vast array of applications, making it highly versatile (unlike

C++ By HAFIZ MUHAMMAD ASAD MUSTAFA

domain-specific languages like SQL). As a compiled language, the source code must be translated into machine-executable binary code by a compiler before the program can be run. This process typically leads to faster execution speeds.

Exemple: C++ is compiled directly into machine code, allowing it to execute instructions with minimal overhead.

Code:

```
// Compilers (like g++) translate this into binary code
int main() { return 0; }
```

Q.2 : What feature allows C++ to be used for systems programming?

Ans: The crucial feature is C++'s ability to provide direct memory manipulation using pointers. This low-level control is essential for tasks like writing operating systems, embedded systems, and device drivers, where managing hardware resources precisely is necessary.

Exemple: Accessing memory addresses directly to modify data outside the normal scope of a function.

Code:

```
int x = 10;
int* ptr = &x; // Get the address of x
*ptr = 20; // Modify the value at that address (x is now 20)
```

Q.3 : Who is credited with developing C++?

Ans: The developer credited with creating C++ (originally named "C with Classes") is Bjarne Stroustrup. He began working on the language in 1979 at Bell Labs, aiming to enhance C with object-oriented features.

Exemple: The core philosophy was to add Simula's object-oriented features to the efficiency of the C language.

Code:

```
// Bjarne Stroustrup's design added classes and objects:
class MyClass { /* ... */};
```

Q.4 : Why is the #include <iostream> directive necessary?

Ans: This is a preprocessor directive that tells the compiler to include the contents of the Input/Output stream library. This library defines essential objects required for performing console input and output operations.

C++ By HAFIZ MUHAMMAD ASAD MUSTAFA

Example: Without including this library, the compiler would not recognize standard I/O objects like cin, cout, and endl.

Code:

```
#include <iostream>
int main() {
    std::cout << "Hello"; // Requires <iostream>
    return 0;
}
```

Q.5 : What is the purpose of the using namespace std; statement?

Ans: The std (standard) namespace contains the names of all entities in the C++ standard library (like cout, cin, string, etc.). This statement brings all those names into the current scope, allowing the programmer to use them without prefixing them with std::.

Example: It simplifies std::cout << "Data"; to just cout << "Data";.

Code:

```
#include <iostream>
using namespace std; // Makes 'cout' available directly

int main() {
    cout << "Standard library element.";
    return 0;
}
```

Q.6 : List the four main sections of a basic C++ program structure.

Ans: A well-structured C++ program generally consists of these four parts, organized sequentially for the compiler:

1. **Preprocessor directives** (#include).
2. **Global declarations** (variables, constants, function prototypes).
3. The **main() function** (the entry point of execution).
4. User-defined functions (the definitions of functions declared in prototypes).
Example: The main() function is always the starting point, and everything else supports its logic.
Code:

```
#include <iostream> // 1. Preprocessor
const int SIZE = 10; // 2. Global Declaration
```

```
int main() { // 3. Main Function
    std::cout << "Program running.";
    return 0;
}
```

Q.7 : Differentiate the primary roles of cout and cin.

Ans: cout (Console Output) is the standard output stream used to write data from the program to the console or screen. cin (Console Input) is the standard input stream used to read data into the program's variables from the keyboard or an input source.

Exemple: cout uses the insertion operator (<<); cin uses the extraction operator (>>).

Code:

```
int age;
cout << "Enter age: "; // Output (cout)
cin >> age;           // Input (cin)
```

Q.8 : What is the significance of the return 0; statement in main()?

Ans: The main() function is expected to return an integer value to the operating system. return 0; is the standard way to indicate that the program has executed successfully without any errors. Any non-zero return value (e.g., return 1;) is traditionally used to signal that an error or failure occurred during execution.

Exemple: If a program fails to open a file, it might return 1; to alert the calling environment (like a shell script).

Code:

```
int main() {
    // Program logic executes here...
    return 0; // Signals success
}
```

Q.9 : What is an I/O Manipulator? Give one example.

Ans: An I/O manipulator is a function or keyword used to control the format, alignment, or precision of data being inserted into or extracted from an I/O stream (cout or cin). They modify the state of the stream until another manipulator overrides the setting.

Exemple: The setprecision(N) manipulator, which controls the number of decimal digits displayed for floating-point values.

Code:

```
#include <iomanip> // Required for setprecision
// ...
cout << setprecision(3) << 12.3456; // Output: 12.3
```

Q.10 : What is a literal? Give an example of a float literal.

Ans: A literal is a fixed value written directly into the source code that cannot be changed. Its type is inferred by its formatting.

Exemple: A float literal must include a decimal point and is explicitly designated as a float (single precision) by appending the suffix f or F.

Code:

```
// 3.14159 is technically a double literal by default
float pi = 3.14159f; // The 'f' suffix makes this a float literal.
```

Q.11 : State the purpose of the fixed manipulator.

Ans: The fixed manipulator is used to force floating-point numbers to be displayed in fixed decimal notation, meaning the output will not use scientific (E) notation. Crucially, when fixed is used, the setprecision manipulator then controls the exact number of digits that appear after the decimal point.

Exemple: Without fixed, setprecision(2) might display only two significant figures (e.g., 1.2e+02).

Code:

```
cout << fixed << setprecision(2) << 123.456; // Output: 123.46
```

Q.12 : Why should you close your input stream with a newline character?

Ans: Using a newline character (\n or endl) ensures that output is moved to the next line, keeping the console tidy. The endl manipulator is often preferred over \n for output streams because it not only inserts a newline but also flushes the output buffer, guaranteeing that the data is immediately sent to the console and displayed to the user.

Exemple: In networked or buffered applications, flushing is vital to ensure message delivery.

Code:

```
// Inserts newline and flushes buffer
```

C++ By HAFIZ MUHAMMAD ASAD MUSTAFA

```
cout << "First line." << endl;  
cout << "Second line.\n"; // Inserts newline only
```

Q.13 : How does the setw() manipulator affect output?

Ans: The setw(N) manipulator sets the minimum field width for the next item inserted into the output stream, where \$N\$ is the total number of characters (including spaces) the output should occupy. If the item is shorter than \$N\$, it is padded with spaces (right-justified by default). It only applies to the next output item.

Exemple: Used for creating neat columns of data.

Code:

```
cout << setw(10) << 100 << setw(10) << 200;  
// Output: (7 spaces)100(7 spaces)200
```

Q.14 : What is the standard library required for string manipulation functions in C-style strings?

Ans: The standard library required for traditional C-style string manipulation (null-terminated character arrays) is <cstring>. This library contains functions inherited from C, such as strlen, strcpy, and strcat.

Exemple: To find the length of a C-style string, you must use strlen(), which is defined in <cstring>.

Code:

```
#include <cstring>  
// ...  
char name[] = "John";  
int len = strlen(name); // len is 4
```

Q.15 : What is the fundamental difference in how char str[] and std::string s store text?

Ans:

- **char str[] (C-style):** Stores text as a **fixed-size array** of characters that **must be null-terminated** (\0). Memory management (size declaration) is manual.
- **std::string s (C++ class):** Stores text as a dynamic object (class) that handles its own memory allocation, resizing, and termination automatically. It offers built-in methods (like .length()).
Exemple: std::string is safer and easier to use for most modern C++ tasks.

Code:

C++ By HAFIZ MUHAMMAD ASAD MUSTAFA

```
char arr[10];    // Fixed size, prone to buffer overflow
std::string text; // Dynamically managed
```

Q.16 : How do you find the length of an std::string object?

Ans: The std::string class provides two identical member functions for querying its length (the number of characters it contains): .length() and .size(). They return the current number of characters in the string.

Exemple: Using the dot operator to call the method on the string object.

Code:

```
std::string data = "CPlusPlus";
int len = data.length(); // len will be 9
```

Q.17 : What are the three essential components of a Function Prototype?

Ans: A function prototype (or function declaration) informs the compiler about the function's interface, allowing it to compile calls to that function before the function itself is defined. The three essential components are:

1. **Return Type** (what data type the function sends back).
2. **Function Name**.
3. **Parameter List** (the data types and sequence of arguments it accepts).

Exemple: The prototype must end with a semicolon ;.

Code:

```
// Return Type, Name, and Parameter List
int calculate_sum(int num1, int num2);
```

Q.18 : Define the Base Case in recursion.

Ans: The Base Case is the critical non-recursive condition within a recursive function. When the function's arguments satisfy this condition, the function returns a value without making any further recursive calls. It serves as the stop condition that prevents the function from calling itself infinitely, thereby avoiding a stack overflow.

Exemple: In the factorial function $n!$, the base case is $1! = 1$ and $0! = 1$.

Code:

```
int factorial(int n) {
```

C++ By HAFIZ MUHAMMAD ASAD MUSTAFA

```
if (n <= 1) { // Base Case
    return 1;
}
return n * factorial(n - 1); // Recursive Call
}
```

Q.19 : What is the benefit of declaring a struct over using separate variables?

Ans: A struct (structure) allows you to group related variables (which can be of different data types) under a single, meaningful name. This greatly improves data organization, readability, and the ability to pass complex bundles of data as a single unit to functions.

Exemple: Grouping a person's name, age, and height into one Person object.

Code:

```
struct Person {
    string name;
    int age;
}; // Groups related data
```

Q.20 : What is the standard way to initialize a std::string object?

Ans: The most common ways are using the assignment operator (copy initialization) or using the constructor (direct initialization) with a string literal.

Exemple: Both methods below achieve the same result.

Code:

```
// 1. Copy Initialization
std::string str1 = "Draft Text";

// 2. Direct Initialization (Constructor call)
std::string str2("Another Draft");
```

Q.21 : List the byte sizes for int, char, and double.

Ans: These sizes are typical on most modern 32-bit/64-bit systems, but can vary by compiler and architecture.

- **int**: Typically **4 bytes** (holding values up to $\approx \pm 2 \times 10^9$).

- **char:** Always **1 byte** (used for ASCII or UTF-8 characters).
- **double:** Typically 8 bytes (double-precision floating-point number).
Example: Knowing the size is important for memory efficiency and pointer arithmetic.
Code:

```
// The sizeof operator confirms these sizes
cout << sizeof(int) << " bytes";
```

Q.22 : Define Implicit Type Casting with an example.

Ans: Implicit Type Casting (or promotion) is an automatic type conversion performed by the compiler without the programmer's explicit request. This typically happens when converting a value from a smaller or less precise type to a larger or more precise type (e.g., int to double) to prevent data loss.

Example: The integer 5 is automatically converted to the double 5.0 before addition.

Code:

```
int i = 5;
double d = 3.5;
double result = i + d; // i is implicitly cast to double
```

Q.23 : Write an example of Explicit Type Casting from double to int.

Ans: Explicit Type Casting is a manual type conversion forced by the programmer. The modern C++ way is to use `static_cast<new_type>(expression)`. When casting a double to an int, the decimal portion is truncated (not rounded).

Example: Truncation of the decimal part will occur.

Code:

```
double price = 19.99;
// Explicitly cast the double to an int
int intPrice = static_cast<int>(price); // intPrice is 19
```

Q.24 : What are the two boolean values represented by the bool type?

Ans: The bool data type represents the two fundamental logical states: true and false. Internally, true is usually stored as the integer value 1, and false as the integer value 0.

Example: Used in conditional statements to control program flow.

Code:

```
bool is_found = true;  
bool is_empty = false;
```

Q.25 : What is the result of the expression 19 / 4 (integer division)?

Ans: Since both operands (19 and 4) are integers, the division operator (/) performs integer division, which calculates the quotient and discards (truncates) the remainder.

Exemple: \$19\$ divided by \$4\$ is \$4\$ with a remainder of \$3\$.

Code:

```
int result = 19 / 4; // result is 4
```

Q.26 : What is the result of 19 % 4?

Ans: The modulo operator (%) calculates the remainder of an integer division.

Exemple: \$19\$ divided by \$4\$ is \$4\$ with \$3\$ left over.

Code:

```
int remainder = 19 % 4; // remainder is 3
```

Q.27 : Differentiate the relational operators == and !=.

Ans: Both are **relational operators** used for comparison, yielding a bool result.

- **== (Equality):** Tests if the value on the left is **equal to** the value on the right.
- **!= (Inequality):** Tests if the value on the left is not equal to the value on the right.
Exemple: Used in loop conditions and if statements.

Code:

```
if (x == 10) { /* ... */ } // If x equals 10  
if (y != 5) { /* ... */ } // If y is not equal to 5
```

Q.28 : How does the Logical OR (||) operator evaluate an expression?

Ans: The Logical OR (||) operator returns true if at least one of its operands evaluates to true. Crucially, it uses short-circuit evaluation: if the left operand is found to be true, the operator immediately returns true and the right operand is never evaluated.

Exemple: This prevents unnecessary function calls or runtime errors.

Code:

```
if (A > 0 || B == 10) { // If A>0 is true, B==10 is skipped
    // ...
}
```

Q.29 : State the order of precedence for the following operators: =, *, +.

Ans: Operator precedence determines which operation is performed first. In C++, multiplication and division (*, /) rank higher than addition and subtraction (+, -), and assignment (=) ranks lowest.

Exemple: The expression $z = x + y * 2$; performs multiplication first.

Code:

```
// Order of Precedence (highest to lowest):
// 1. * (Multiplication)
// 2. + (Addition)
// 3. = (Assignment)
```

Q.30 : What is the shorthand equivalent for the expression $x = x + 5$;

Ans: This expression can be simplified using the compound assignment operator for addition, which combines the operation and the assignment in one symbol.

Exemple: This is concise and often results in slightly more optimized machine code.

Code:

```
// Shorthand equivalent:
x += 5;
```

Q.31 : What is the Unary Operator version of the assignment operator?

C++ By HAFIZ MUHAMMAD ASAD MUSTAFA

Ans: The unary operators that perform an implicit assignment are the increment operator (++) and the decrement operator (--). They modify the value of a single variable by adding or subtracting one.

Exemple: Used extensively in loop counters.

Code:

```
int count = 0;
count++; // Equivalent to count = count + 1;
```

Q.32 : What will the value of x be after `int x = 5; x++;`?

Ans: The post-increment operator (x++) increases the value of x by 1. Since the operator is used as a standalone statement, the final value of x will be 6.

Exemple: If x++ were part of a larger expression, the original value of x would be used before the increment.

Code:

```
int x = 5;
x++;
// x is now 6
```

Q.33 : How do you initialize a variable to ensure it holds a garbage value?

Ans: A variable holds a garbage value (unpredictable, leftover data from the memory location) if it is simply declared without being explicitly initialized by the programmer. This is typically true for local variables (those declared inside a function).

Exemple: Accessing this variable before assignment leads to undefined behavior.

Code:

```
int uninitialized_var;
// uninitialized_var holds a garbage value.
```

Q.34 : What are the common names for the operators + and - when used on a single variable?

Ans: When these operators are used on a single operand to denote sign or negation, they are called Unary plus (+) and Unary minus (-) operators.

Exemple: The unary minus operator negates the value of the variable.

Code:

```
int num = 10;  
int neg = -num; // Unary minus: neg is -10
```

Q.35 : How is the expression $A = B$ evaluated by the processor?

Ans: The assignment operation reads the value stored in the memory location associated with variable B, and then copies that value into the memory location designated for variable A. The value of A is thus overwritten by the value of B.

Exemple: `int x = 10; int y = 20; x = y;` (x becomes 20).

Code:

```
int x = 10;  
int y = 20;  
x = y; // x receives a copy of y's value
```

Q.36 : Why is a switch statement generally faster than a chain of if-else if statements?

Ans: The compiler can often implement a switch statement using an optimized technique called a jump table. This allows the program to calculate an index and jump directly to the correct case block in constant time $O(1)$, whereas an if-else if chain must check each condition sequentially, which is proportional to the number of checks $O(n)$.

Exemple: A switch statement with 10 cases finds the correct case instantly, while an if-else if chain might check up to 10 conditions.

Code:

```
switch(choice) {  
    case 1: // Direct jump  
        // ...  
}
```

Q.37 : What is the only circumstance in which a do-while loop will execute zero times?

Ans: A do-while loop is characterized by its post-test condition, meaning the condition is checked after the loop body executes. Therefore, a do-while loop is guaranteed to execute at least once; it can never execute zero times, regardless of the initial condition.

Exemple: Used when input from the user must be read before the exit condition is checked.

Code:

```
int i = 10;
do {
    // This code runs at least once
} while (i < 5); // Condition is false, but runs once.
```

Q.38 : When should the break statement be used in a non-switch context?

Ans: In a non-switch context (i.e., inside a for, while, or do-while loop), the break statement is used to immediately and completely terminate the loop. Execution continues with the statement immediately following the loop body.

Exemple: Used when a required item is found in a search, making further iteration unnecessary.

Code:

```
for (int i = 0; i < 100; i++) {
    if (i == 50) {
        break; // Exit loop immediately
    }
}
```

Q.39 : What is the effect of the continue statement?

Ans: The continue statement skips the remainder of the code block in the current iteration of the loop. It immediately jumps to the next iteration (for while loops, this means jumping to the condition check; for for loops, it jumps to the update expression).

Exemple: Used to skip processing steps for specific values without breaking the entire loop.

Code:

```
for (int i = 0; i < 5; i++) {
    if (i == 2) {
        continue; // Skip printing 2
    }
    cout << i;
} // Output: 0134
```

Q.40 : How does the body of a while loop differ from that of a for loop?

Ans: In a while loop, the iteration or update step (e.g., the counter increment like i++ or count--) must be explicitly

C++ By HAFIZ MUHAMMAD ASAD MUSTAFA

written and managed inside the loop's body. In a for loop, the update step is defined in the loop header, ensuring it is executed automatically.

Exemple: Forgetting the update step in a while loop results in an infinite loop.

Code:

```
int i = 0;
while (i < 5) {
    cout << i;
    i++; // Must be explicit here
}
```

Q.41 : What determines the number of times a recursive function is called?

Ans: The number of recursive calls is determined by the Base Case and the way the input parameters are modified in each recursive call. The modifications must guarantee that the parameters eventually satisfy the base case condition, leading to the termination of the recursive chain.

Exemple: In factorial(n), n must decrease by 1 in each call to eventually reach the base case $n \leq 1$.

Code:

```
// The decrease (n-1) moves closer to the base case (1)
return n * factorial(n - 1);
```

Q.42 : What is the index of the first element in any C++ array?

Ans: C++ uses zero-based indexing for arrays (inherited from C). Therefore, the index of the first element in any array is 0.

Exemple: If an array has 5 elements, they are indexed from 0 to 4.

Code:

```
int arr[5] = {1, 2, 3, 4, 5};
cout << arr[0]; // Output: 1
```

Q.43 : If an array is declared as float grades[25];, what is the valid range of indices?

Ans: Since the array size is 25 and indexing starts at 0, the valid indices range from 0 through 24 (inclusive). Accessing grades[25] or any index outside this range constitutes out-of-bounds access and leads to undefined

behavior.

Exemple: Using grades[24] is valid; grades[-1] is not.

Code:

```
float grades[25];  
grades[24] = 95.5f; // Valid
```

Q.44 : Name the operator used to retrieve the memory address of a variable.

Ans: The Address-of operator (&). This unary operator returns the memory location where the variable is stored, and the result is a pointer to that type.

Exemple: Used to initialize a pointer variable.

Code:

```
int value = 50;  
int* ptr = &value; // ptr stores the address of value
```

Q.45 : Name the operator used to access the value stored at a pointer's address.

Ans: The Dereference operator or Indirection operator (*). This unary operator accesses the actual value stored in the memory location that the pointer is holding an address for.

Exemple: Dereferencing a pointer to read or write the data it points to.

Code:

```
// Assuming ptr points to 50  
cout << *ptr; // Output: 50
```

Q.46 : If ptr is an int* pointing to the first element of an array, what does ptr + 1 represent?

Ans: It represents the memory address of the second element of the array (arr[1]). Due to pointer arithmetic, adding 1 to the pointer doesn't just add 1 byte; it adds 1 * sizeof(int) bytes, moving the pointer exactly to the start of the next integer element.

Exemple: If sizeof(int) is 4 bytes, ptr + 1 is 4 bytes ahead of ptr.

Code:

```
int arr[] = {10, 20};  
int* ptr = arr;  
// *(ptr + 1) accesses 20
```

Q.47 : How do you check if a pointer is safe to use?

Ans: Before dereferencing a pointer, you must check if it points to a valid memory location. The standard way to check is to ensure the pointer is not equal to the null pointer (nullptr). A null pointer indicates that the pointer is intentionally not pointing to any object or address.

Exemple: Dereferencing a nullptr results in a crash (segmentation fault).

Code:

```
if (ptr != nullptr) {  
    // safe to use *ptr  
}
```

Q.48 : What is the meaning of the array declaration int matrix[5][10];?

Ans: This declaration defines a two-dimensional array of integers named matrix. The first dimension [5] specifies that it has 5 rows, and the second dimension [10] specifies that it has 10 columns. It holds a total of $5 \times 10 = 50$ integer elements.

Exemple: matrix[0][0] is the top-left element; matrix[4][9] is the bottom-right element.

Code:

```
// Accessing the element in the 3rd row (index 2), 5th column (index 4)  
matrix[2][4] = 99;
```

Q.49 : Why does C++ not require the size of the first dimension of a 2D array when passing it to a function?

Ans: When passed to a function, a 2D array decays into a pointer to its first row: int (*arr)[10]. The compiler only needs to know the size of the subsequent dimensions (columns) to correctly calculate the memory offset when accessing elements like arr[i][j]. The size of the first dimension (rows) is irrelevant for this calculation.

Exemple: The function signature must include the column size: void printMatrix(int arr[][10]).

Code:

```
void func(int arr[][10]) { /* ... */ }
```

// The 'l0' is necessary for addressing.

Q.50 : What does the term pointer arithmetic refer to?

Ans: Pointer arithmetic is performing addition or subtraction operations on a pointer variable. The key is that the arithmetic is automatically scaled by the size of the data type the pointer points to. Adding $\$N\$$ to a pointer moves the address by $\$N\$$ multiples of the data type's size, effectively moving the pointer to the $\$N^{\text{th}}\$$ next element.

Exemple: `ptr++` moves the pointer to the start of the next element, regardless of whether that element is 1 byte (char) or 8 bytes (double).

Code:

```
double* dPtr; // points to a double
dPtr = dPtr + 1; // Address jumps by 8 bytes (assuming sizeof(double) is 8)
```

Long Questions:

Q.1 : Write a C++ program that calculates the total cost of items. It should apply a 15% discount if the total cost exceeds \$500, and then apply a fixed 5% sales tax to the final amount.

Ans: This program demonstrates basic input, arithmetic, and the use of the if statement for conditional logic. It also uses I/O manipulators (fixed and setprecision) to format the final currency output correctly.

Exemple: If the price is 100 and quantity is 6, the subtotal is 600. Discount applied (15% of 600 = 90). New total is 510. Tax applied (5% of 510 = 25.50). Final price is 535.50.

Code:

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double price, quantity, total;

    cout << "Enter price per item: $";
    cin >> price;
    cout << "Enter quantity: ";
    cin >> quantity;

    total = price * quantity;

    // Check for discount condition
    if (total > 500) {
```

C++ By HAFIZ MUHAMMAD ASAD MUSTAFA

```
    cout << "--- 15% bulk discount applied! ---" << endl;
    total *= 0.85; // total = total * (1 - 0.15)
}

// Apply 5% sales tax to the current total
total *= 1.05; // total = total * (1 + 0.05)

cout << fixed << setprecision(2);
cout << "Final price (including tax): $" << total << endl;

return 0;
}
```

Q.2 : Write a C++ program that takes three side lengths of a triangle as input and determines whether the triangle is Equilateral, Isosceles, or Scalene. Include a check for validity.

Ans: The solution uses nested if-else if-else statements. First, it uses the Triangle Inequality Theorem ($a+b>c$, $a+c>b$, $b+c>a$) to ensure the sides form a valid triangle. Then, it checks for equality conditions: three equal sides (Equilateral), two equal sides (Isosceles), or no equal sides (Scalene).

Exemple: Input: 5, 5, 5 -> Equilateral. Input: 4, 4, 6 -> Isosceles. Input: 1, 2, 9 -> Invalid.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int a, b, c;

    cout << "Enter three side lengths of triangle: ";
    cin >> a >> b >> c;

    // Check triangle validity first
    if (a + b > c && a + c > b && b + c > a) {
        if (a == b && b == c) {
            cout << "Equilateral triangle" << endl;
        }
        else if (a == b || a == c || b == c) {
            cout << "Isosceles triangle" << endl;
        }
        else {
            cout << "Scalene triangle" << endl;
        }
    } else {
        cout << "Invalid triangle sides! The sum of any two sides must be greater than the third." << endl;
    }
}
```

```
    return 0;  
}
```

Q.3 : Write a program that accepts a single character input and uses a switch statement to determine if it is a vowel, a consonant, or not a letter.

Ans: This program demonstrates the switch statement's efficiency for checking multiple discrete values. To handle case-insensitivity, the program uses `tolower()` from the `<cctype>` library to convert the input to lowercase before checking the vowel cases. The `isalpha()` function is used in the default block to distinguish consonants from non-letter characters.

Exemple: Input: 'A' -> vowel. Input: 'b' -> consonant. Input: '9' -> not a letter.

Code:

```
#include <iostream>  
#include <cctype>  
using namespace std;  
  
int main() {  
    char ch;  
  
    cout << "Enter a single character: ";  
    cin >> ch;  
  
    // Convert to lowercase for simplified checking  
    char lowerCh = tolower(ch);  
  
    switch(lowerCh) {  
        case 'a':  
        case 'e':  
        case 'i':  
        case 'o':  
        case 'u':  
            cout << "" << ch << " is a vowel" << endl;  
            break;  
        default:  
            // If it wasn't a vowel, check if it's an alphabet character  
            if (isalpha(ch)) {  
                cout << "" << ch << " is a consonant" << endl;  
            } else {  
                cout << "" << ch << " is not a letter" << endl;  
            }  
        }  
  
    return 0;  
}
```

Q.4 : Write a program that determines if a positive integer entered by the user is a prime number.

Ans: A prime number is an integer greater than 1 that has no positive divisors other than 1 and itself. This program uses a for loop to check for divisibility. For efficiency, the loop only iterates from $i=2$ up to $\sqrt{n}$. If any divisor is found, a bool flag (isPrime) is set to false, and the loop is immediately terminated using break.

Exemple: Input: 7 -> Prime. Input: 9 -> Not Prime (divisible by 3). Input: 1 -> Not Prime.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    bool isPrime = true;

    cout << "Enter a positive integer: ";
    if (!(cin >> n) || n <= 0) {
        cout << "Invalid input. Must be a positive integer." << endl;
        return 1;
    }

    if (n == 1) {
        isPrime = false;
    } else {
        // Check divisibility from 2 up to the square root of n
        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                isPrime = false;
                break; // Optimization: Exit once a factor is found
            }
        }
    }

    if (isPrime) {
        cout << n << " is a prime number" << endl;
    } else {
        cout << n << " is not a prime number" << endl;
    }

    return 0;
}
```

Q.5 : Write a program that uses a while loop to count backward from 20 to 1, skipping and not printing any number that is a multiple of 5.

Ans: This program uses a while loop to control iteration. Inside the loop, an if statement checks if the counter (i) is divisible by 5 using the modulo operator ($i \% 5 == 0$). If it is, the counter is decremented, and the continue statement is executed, skipping the cout statement and moving immediately to the next iteration.

Exemple: Output will be: 20 is skipped, 15 is skipped, 10 is skipped, 5 is skipped. Printed numbers: 19 18 17 16 14 13 12 11 9 8 7 6 4 3 2 1.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int i = 20;

    cout << "Counting down from 20 to 1 (skipping multiples of 5):" << endl;

    while (i >= 1) {
        // If it's a multiple of 5, skip the printing
        if (i % 5 == 0) {
            i--; // Decrement before continuing!
            continue;
        }
        cout << i << " ";
        i--;
    }
    cout << endl;

    return 0;
}
```

Q.6 : Explain and provide code contrasting function parameter passing mechanisms: Pass by Value and Pass by Reference.

Ans:

- **Pass by Value:** A copy of the argument's value is made and passed to the function. Any modification inside the function affects only this copy, leaving the original variable in the calling scope unchanged.
- **Pass by Reference:** An alias (reference, denoted by &) to the original variable's memory location is passed. Modifications inside the function directly change the value of the original variable.
Exemple: The value of num changes after incrementReference but not after incrementValue.

Code:

```
#include <iostream>
using namespace std;
```

C++ By HAFIZ MUHAMMAD ASAD MUSTAFA

```
// Pass by Value: takes a copy (original not changed)
void incrementValue(int x) {
    x++;
    cout << "Inside Value function: " << x << endl; // Incremented copy
}

// Pass by Reference: takes an alias (original is changed)
void incrementReference(int &x) {
    x++;
    cout << "Inside Reference function: " << x << endl; // Incremented original
}

int main() {
    int num = 5;

    cout << "Original value: " << num << endl; // 5

    incrementValue(num);
    cout << "After pass by value: " << num << endl; // Still 5

    incrementReference(num);
    cout << "After pass by reference: " << num << endl; // Now 6

    return 0;
}
```

Q.7 : Write three overloaded functions named calculateArea that compute the area of a square (int), a rectangle (two doubles), and a circle (one double).

Ans: Function overloading allows multiple functions to share the same name provided they have distinct parameter lists (different number or types of parameters). The compiler determines which function to call at compile time based on the arguments provided. This improves code usability and clarity.

Exemple: Calling calculateArea(5) executes the integer version; calculateArea(4.5, 3.2) executes the two-double version.

Code:

```
#include <iostream>
using namespace std;

const double PI = 3.14159;

// Overload 1: Area of square (integer side)
int calculateArea(int side) {
    return side * side;
}
```

```
// Overload 2: Area of rectangle (double length, double width)
double calculateArea(double length, double width) {
    return length * width;
}

// Overload 3: Area of circle (double radius)
double calculateArea(double radius) {
    return PI * radius * radius;
}

int main() {
    // Compiler selects the correct function automatically:
    cout << "Area of square (side 5): " << calculateArea(5) << endl;
    cout << "Area of rectangle (4.5 x 3.2): " << calculateArea(4.5, 3.2) << endl;
    cout << "Area of circle (radius 2.5): " << calculateArea(2.5) << endl;

    return 0;
}
```

Q.8 : Implement a recursive function to compute the N^{th} number in the Fibonacci sequence.

Ans: The Fibonacci sequence is naturally recursive: $F(n) = F(n-1) + F(n-2)$. The function solves the problem by breaking it down into two smaller sub-problems. The critical part is defining the two base cases: $F(0) = 0$ and $F(1) = 1$, which stops the recursion.

Exemple: Input: 6. Output: 8 (Sequence: 0, 1, 1, 2, 3, 5, 8...).

Code:

```
#include <iostream>
using namespace std;

int fibonacci(int n) {
    // Base cases (Stop conditions)
    if (n <= 0) return 0;
    if (n == 1) return 1;

    // Recursive case: Sum of the two preceding numbers
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int n;

    cout << "Enter position (N) in Fibonacci sequence: ";
    cin >> n;
```

```
if (n < 0) {
    cout << "Please enter a non-negative number." << endl;
} else {
    cout << "The Fibonacci number at position " << n << " is: " << fibonacci(n) << endl;
}

return 0;
}
```

Q.9 : Write a program that uses two separate functions to find the maximum and minimum elements in a pre-defined integer array.

Ans: This program demonstrates how arrays are passed to functions (they decay into pointers to their first element) and how to iterate through the elements to find the largest and smallest values. Both functions initialize their respective tracking variables (max and min) with the first element of the array.

Exemple: For the array {23, 45, 12, 67, 89}, the max is 89 and the min is 12.

Code:

```
#include <iostream>
using namespace std;

// Function to find maximum element in array
int findMax(int arr[], int size) {
    int max = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}

// Function to find minimum element in array
int findMin(int arr[], int size) {
    int min = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] < min) {
            min = arr[i];
        }
    }
    return min;
}

int main() {
    const int SIZE = 10;
    int numbers[SIZE] = {23, 45, 12, 67, 89, 34, 56, 78, 90, 11};
```

```
cout << "Maximum element: " << findMax(numbers, SIZE) << endl;
cout << "Minimum element: " << findMin(numbers, SIZE) << endl;

return 0;
}
```

Q.10 : Write a program to calculate the sum of all elements in a 3x3 two-dimensional array.

Ans: Summing the elements of a 2D array requires nested for loops. The outer loop iterates through the rows, and the inner loop iterates through the columns for the current row. The value of each element `matrix[i][j]` is added to a running sum variable.

Exemple: For a 3x3 matrix initialized with 1 to 9, the sum is $1+2+...+9 = 45$.

Code:

```
#include <iostream>
using namespace std;

int main() {
    const int ROWS = 3;
    const int COLS = 3;
    int matrix[ROWS][COLS] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    int sum = 0;

    // Nested loops for 2D array traversal
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            sum += matrix[i][j];
        }
    }

    cout << "Sum of all elements in the 3x3 matrix: " << sum << endl;

    return 0;
}
```

Q.11 : Write a function that accepts two integer pointers and swaps the values that they point to. Demonstrate

the use in main.

Ans: This task illustrates the fundamental use of pointers for modifying variables outside the function's scope. The function receives the addresses of two variables. It must use the dereference operator (*) to access the values stored at those addresses to perform the actual swap, requiring a temporary variable for storage.

Exemple: If a=5 and b=10 are passed, the function swaps their original values in memory, resulting in a=10 and b=5.

Code:

```
#include <iostream>
using namespace std;

// Function to swap values using pointers
void swapPointers(int *ptr1, int *ptr2) {
    // Dereference both pointers to access their values
    int temp = *ptr1;
    *ptr1 = *ptr2;
    *ptr2 = temp;
}

int main() {
    int a = 5, b = 10;

    cout << "Before swap: a = " << a << ", b = " << b << endl;

    // Pass the addresses of a and b
    swapPointers(&a, &b);

    cout << "After swap: a = " << a << ", b = " << b << endl;

    return 0;
}
```

Q.12 : Write a program to iterate through a single-dimensional integer array using only pointer arithmetic.

Ans: This program demonstrates array traversal without using the array subscript operator ([]). The array name itself acts as a constant pointer to the first element. We use a pointer variable ptr initialized to the array's start. In the loop, the expression *(ptr + i) is used, where the pointer arithmetic ptr + i calculates the address of the i^{th} element, and * dereferences it to retrieve the value.

Exemple: The loop will print 10, 20, 30, 40, 50.

Code:

```
#include <iostream>
using namespace std;

int main() {
```

C++ By HAFIZ MUHAMMAD ASAD MUSTAFA

```
const int SIZE = 5;
int arr[SIZE] = {10, 20, 30, 40, 50};
int *ptr = arr; // Pointer points to first element (address of arr[0])

cout << "Array elements accessed via pointer arithmetic:" << endl;

// Loop uses index 'i' but accesses memory via *(ptr + i)
for (int i = 0; i < SIZE; i++) {
    cout << "Element " << i << ": " << *(ptr + i) << endl;
}

return 0;
}
```

Q.13 : Define a struct named Student with fields for name (string), ID (int), and GPA (float). Write a program to input and display one student's data.

Ans: A struct is defined globally to aggregate the data fields. In main, an instance of the Student struct is created. Data is then input from the console and assigned to the individual members of the struct using the dot operator (.).

Exemple: Allows a single variable (student1) to hold three pieces of related information.

Code:

```
#include <iostream>
#include <string>
using namespace std;

// Define the Student structure
struct Student {
    string name;
    int ID;
    float GPA;
};

int main() {
    Student student1;

    // Input student data using the dot operator
    cout << "Enter student Name: ";
    cin >> student1.name;
    cout << "Enter student ID: ";
    cin >> student1.ID;
    cout << "Enter student GPA: ";
    cin >> student1.GPA;

    // Display student data
    cout << "\n--- Student Record ---" << endl;
```

C++ By HAFIZ MUHAMMAD ASAD MUSTAFA

```
cout << "Name: " << student1.name << endl;
cout << "ID: " << student1.ID << endl;
cout << "GPA: " << student1.GPA << endl;

return 0;
}
```

Q.14 : Write a program that accepts a sentence and performs three operations: prints its total length, prints it in uppercase, and prints a substring of characters from position 3 to 8 (inclusive).

Ans: This program uses the powerful std::string class features. It uses getline() to read the full sentence (including spaces). The .length() method retrieves the size. A range-based for loop combined with toupper() prints the uppercase version. Finally, the .substr(index, length) method is used for extraction.

Exemple: Input: "Learning C++ is fun". Output length 19. Substring "rning ".

Code:

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

int main() {
    string sentence;

    cout << "Enter a sentence: ";
    getline(cin, sentence); // Reads the whole line

    // 1. Print total length
    cout << "\n1. Total length: " << sentence.length() << " characters" << endl;

    // 2. Print in uppercase (using range-based for loop)
    cout << "2. Uppercase: ";
    for (char c : sentence) {
        cout << (char)toupper(c);
    }
    cout << endl;

    // 3. Print substring (position 3 is index 2; length of 6 characters)
    if (sentence.length() >= 9) {
        // .substr(starting_index, length_to_extract)
        cout << "3. Substring (chars 3-8): " << sentence.substr(2, 6) << " " << endl;
    } else {
        cout << "3. String too short for required substring operation." << endl;
    }

    return 0;
}
```

```
}
```

Q.15 : Using the Student struct from Q.13, create an instance and a pointer to it. Demonstrate accessing and modifying the struct members using both the dereference (*) and arrow (->) operators.

Ans: This task demonstrates how to work with complex data types via pointers. The pointer is initialized with the address of the struct instance. Members are accessed: 1) by explicitly dereferencing the pointer first ((*sPtr).name) or 2) by using the convenient arrow operator (->), which is syntactic sugar for the dereference-dot sequence.

Exemple: The GPA is modified from 3.8 to 3.9 using the arrow operator.

Code:

```
#include <iostream>
#include <string>
using namespace std;

struct Student {
    string name;
    int ID;
    float GPA;
};

int main() {
    // Create and initialize a student instance
    Student student1 = {"Alice Johnson", 1001, 3.8f};

    // Declare a pointer to the Student structure
    Student *sPtr = &student1;

    cout << "Accessing structure members through pointer:" << endl;

    // 1. Using the Arrow operator (Preferred method)
    cout << "Name (->): " << sPtr->name << endl;

    // 2. Using the Dereference and Dot operator
    cout << "GPA (*. ): " << (*sPtr).GPA << endl;

    // Modify through the pointer
    sPtr->GPA = 3.9f;
    cout << "\nUpdated GPA (via pointer): " << student1.GPA << endl;

    return 0;
}
```

The End

Thank you for reading these notes. If you find them helpful, **share them with others**, because *sharing is caring* and **knowledge grows when it is shared**.

If you have any questions, suggestions, or need further assistance, feel free to contact me.

Contact Information:

Email: hafizasad2554@gmail.com

Phone: 0303-2554632